

10. Templates

What

- Templates in C++ are a **compile-time mechanism** that lets you write **generic code** once and use it with many different types, allowing the compiler to generate type-specific versions automatically.
- we write code without writing code
- **compile-time code generators**.

```
1 //for int
2 int min(int b,int a){
3     return ( b>a ) ? a : b ;
4 }
5
6 //for double
7 double min(double b,double a){
8     return ( b>a ) ? a : b ;
9 }
10
11 //for float
12 float min(float b,float a){
13     return ( b>a ) ? a : b ;
14 }
```

Note

Logic is the same all along

We need to recreate the functions because we need to add a variable for datatype

idea is like

a variable that stores the type of variable

:- variable_to_store_data_type :- Templates

```
1 template<typename T>
2 T min(T b,T a){
3     return ( b>a ) ? a : b ;
4 }
5 cout<<min(2,3); // 2
6 cout<<min(2.1,3); //2.1
```

```
7 cout<<min(2.45,3.333); //3.333
8 cout<<min("Medhansh","Someone"); //Medhansh
9 cout<<min('a','b'); // a
```

- in languages like JAVA :- these are called generics (i guess)

Two ways to call a template

Explicit type

```
1 max<int>(10, 20);
```

You tell the compiler the type.

Type deduction

```
1 max(10, 20);
```

The compiler figures out `T` automatically.

This is called **template argument deduction**.